

**Esercizi per il corso di Data Science - Laurea in Scienza dei Materiali**

PROF. D. DI SANTE, DR. A. CONSIGLIO  
SEMESTRE INVERNALE 2025/2026

**3° Foglio, Regressione Lineare e PCA**  
12/11/2022

**Esercizio 1 - Ampiezza della banda proibita in composti binari**

Vogliamo predire il *band gap* (banda proibita) calcolato di composti binari del database Materials Project (MP), usando soltanto 10 features rese disponibili da MP.

Le dieci features da utilizzare sono (usando la API di MP):

1. `formation_energy_per_atom` [eV/atom]
2. `energy_above_hull` [eV/atom]
3. `density` [g/cm<sup>3</sup>]
4. `volume` [ang<sup>3</sup> per cella]
5. `nsites` [numero di atomi per cella]
6. `symmetry.number` [numero di gruppo spaziale]
7. `efermi` [eV]
8. `total_magnetization` [ $\mu_B$  per cella]
9. `total_magnetization_normalized_vol` [ $\mu_B$ /ang<sup>3</sup>]
10. `num_magnetic_sites` [conteggio siti magnetici]

Per identificare il materiale in tabella, conservate anche `material_id` e `formula_pretty` (non usatele come feature).

La variabile da predire è `band_gap` [eV] restituita dal *Summary endpoint* di MP. Non deve essere inclusa tra le feature di input.

Si vincoli la ricerca ai composti binari, ovvero quelli formati solo da due elementi chimici (`nelements` = 2).

Il modello base è

$$\hat{y}_i = \beta_0 + \sum_{j=1}^{10} \beta_j \tilde{x}_{ij},$$

dove  $y_i$  è il `band_gap` in eV.

- ~~Quali feature mostrano la correlazione più forte con il *band gap*?~~

- Dove il modello sbaglia di più (materiali ad alto o basso gap)?
- La regolarizzazione migliora la generalizzazione? Come variano i coefficienti tra Ridge e Lasso?
- Come cambia il problema utilizzando una regressione polinomiale?

## Esercizio 2 - Metodo PCA per la banca dati MNIST

Il problema di classificazione MNIST è uno dei classici problemi di apprendimento automatico per la classificazione di dati ad alta dimensione con un numero elevato di esempi. In questa banca dati, ogni cifra scritta a mano è disponibile in un'immagine quadrata in scala di grigi sotto forma di una griglia di pixel da  $28 \times 28$ . Ogni pixel assume un valore compreso nell'intervallo  $[0, 255]$ , che rappresenta una delle 256 sfumature del colore grigio.

(a) Si scarichi il dataset MNIST, che contiene immagini di cifre (0-9) scritte a mano in formato  $28 \times 28$  pixel, e lo si trasformi in un vettore di 784 ( $28 \times 28$ ) caratteristiche per ogni immagine.

Per scaricare i dati MNIST da <https://www.openml.org/d/554> è possibile utilizzare il comando `fetch_openml` di `sklearn.datasets`:

```
X, y = fetch_openml('mnist_784', version=1, return_X_y=True)
```

(b) Si mescolino i dati in ordine casuale per garantire che il training e il test set siano bilanciati e si selezionino 50.000 campioni come insieme di training e 10.000 come insieme di test.

(c) Si applichi 'StandardScaler' per normalizzare le caratteristiche, in modo che abbiano media zero e varianza unitaria, condizione importante per la convergenza dell'algoritmo di regressione.

(d) Si crei e si addestrì un modello di regressione logistica utilizzando il metodo del gradiente stocastico (sag) per l'ottimizzazione.

(e) Per valutare il modello si calcoli la sparsità, ossia la percentuale di pesi non nulli. Si calcoli inoltre l'accuratezza del modello sull'insieme di test.

(f) Si visualizzino ora i pesi appresi dal modello, per comprendere come il modello distingue le diverse cifre.

Per fare ciò si copino i pesi dal modello per poterli modificare senza alterare il modello originale. Successivamente si calcoli la scala per visualizzare i pesi come immagini, determinando il valore massimo da usare per normalizzare la visualizzazione dei pesi (`scale = np.abs(coef).max()`).

Per ciascuna delle 10 classi di cifre (0-9), crea un sottografico visualizzando i pesi come un'immagine  $28 \times 28$ , che rappresenta i pesi del modello come intensità di pixel.

(g) Si applichi l'Analisi delle Componenti Principali (PCA) per riuscire a ottenere un'immagine, utilizzando una frazione del numero di dimensioni (ovvero significativamente inferiore delle 784 variabili pixel).

(h) Si mostri quanta varianza nel set di dati originale è spiegata dalle prime  $k$  componenti principali.

(i) Si utilizzi la PCA per visualizzare i dati originali ad alta dimensionalità in due dimensioni, per vedere se eventuali agglomerati di dati sono chiaramente visibili.

### **Esercizio 3 - Auto-facce e algoritmo PCA**

Si applichi il metodo delle autofacce (eigenfaces) per rappresentare una foto in primo piano di un volto come combinazione lineare delle autofacce estratte da un set di dati.

Si utilizzino le tecniche di riduzione della dimensionalità come l'Analisi delle Componenti Principali (PCA) per svolgere l'esercizio.

(a) Per prima cosa si recuperi il dataset di volti fornito su Virtuale, comprendendo il formato delle immagini e la loro organizzazione, aiutandovi con il notebook jupyter.

(b) Caricate il dataset e appiattite le immagini da matrici a vettori.

(c) Si utilizzi la PCA per decomporre il dataset in autofacce. Successivamente, visualizzate le prime  $k$  autofacce per interpretarne il significato.

(d) Si carichi una nuova foto di un volto in primo piano. Per prima cosa, la si renda compatibile con la base della banca dati.

Successivamente, proiettate l'immagine nello spazio delle autofacce per calcolare i coefficienti di espansione lineare. Ricostruite l'immagine a partire da un numero  $k$  di autofacce e confrontatela con l'immagine originale. Variate  $k$  per analizzare l'accuratezza della ricostruzione.

(e) Valutate la perdita di informazione in base al numero di autofacce utilizzate. Create un grafico che mostri la differenza tra immagine originale e ricostruzione al variare di  $k$ .

(f) Si espanda l'immagine di un altro (s)oggetto, ad esempio una pianta, nella base delle autofacce. Quante autofacce sono necessarie per una rappresentazione accurata?